Completado

S1 · Paso 1 de 22

Crear estructura del Skill

Tiempo estimado: 5 minutos. Crea el Skill en la ruta estandar `.github/skills/` de `workshop-auth-backend`.

```
mkdir -p .github/skills/feature-intake-spec-checklist/examples
```

```
touch .github/skills/feature-intake-spec-checklist/SKILL.md
```

```
touch .github/skills/feature-intake-spec-
checklist/examples/input.sample.md
```

```
touch .github/skills/feature-intake-spec-
checklist/examples/output.sample.md
```

Salida esperada: En Bash, los comandos no muestran mensajes. Crean la carpeta y los tres archivos inicialmente vacíos; la estructura resultante es:

```text
.github/
└── skills/
    └── feature-intake-spec-checklist/
        ├── SKILL.md
        └── examples/
            ├── input.sample.md
            └── output.sample.md
```

Completado

S1 · Paso 2 de 22

Completar el contrato

Tiempo estimado: 30-40 minutos. Abre el `SKILL.md` vacío que creaste en el paso anterior y pega la plantilla completa. Completa los campos que la plantilla deje pendientes para tu caso; no contiene marcadores [EDITAR].

Texto

```
---
name: feature-intake-spec-checklist
description: Estandariza la definición de funcionalidades antes de la
implementación generando un paquete mínimo trazable que cubra alcance,
decisiones, riesgos, criterios de aceptación y planes de validación.
---

# Skill: Feature Intake -> Spec Checklist

## 1) Metadata
- **Skill ID:** feature-intake-spec-checklist
- **Version:** 0.1.0
- **Purpose:** Estandarizar la definición de funcionalidades antes de la
implementación generando un paquete mínimo trazable que cubra alcance,
decisiones, riesgos, aceptación y validación.
- **Audience (roles):** Chapter/PO, Tech Lead, Backend, Frontend, QA, Cyber
- **When to use:** Antes de crear tareas de implementación o abrir un PR
significativo.
- **When NOT to use:** Incidentes P1 de producción, cambios triviales como
typos o tareas sin impacto funcional.
- **Sensitive inputs:** No incluyas datos reales de clientes, credenciales,
secretos, tokens ni URLs internas no autorizadas.
- **Expected output:** Markdown estructurado mediante la sección Output a
continuación.

## 2) Inputs
```

Criterio de salida: otro rol debe poder leer el archivo y saber que activa el Skill, que informacion recibe, que devuelve, que limites respeta y como se valida.

Completado

S1 · Paso 3 de 22

Crear entrada

Tiempo estimado: 5-10 minutos. Pega el caso de ejemplo en `examples/input.sample.md`; puedes adaptar contexto, impacto o dependencias, conservando los ocho encabezados.

Texto

```
## Business context (brief)
Reduce complaints caused by duplicate transfers.

## Feature request / user story
As a user, I want to confirm a transfer before sending it
so that I can avoid duplicates.

## Expected impact
Reduce duplicate-transfer complaints by 20% in one month.

## Scope (in/out)
In: pre-send confirmation, summary message, data validation.
Out: infrastructure changes and a complete UI redesign.

## Known technical constraints
Existing REST API, web frontend, standardized logging.

## Known risks
Sensitive data exposure in logs; amount-validation errors.

## Dependencies
Accounts service and transfers service.

## Team Definition of Done (DoD)
Unit tests, brief documentation, and reviewed PR.
```

Criterio de salida: la entrada es suficientemente concreta para generar una especificacion sin inventar requisitos; lo no confirmado queda como pregunta abierta.

Completado

S1 · Paso 4 de 22

Crear salida

Tiempo estimado: 10-15 minutos. Genera `examples/output.sample.md` a partir de la entrada; puedes cambiar decisiones, riesgos o tareas pero no elimines secciones.

Texto

```
# Feature Intake -> Spec Checklist

## 1. Summary
Add a pre-send transfer confirmation with a concise summary
and validation of transfer data. Target: 20% reduction in
duplicate-transfer complaints in one month.

## 6. Acceptance criteria (measurable)
- 95%+ of valid attempts show confirmation before submission.
- Invalid data blocks submission with a validation message.
- Duplicate complaints drop 20% within one month.
- No sensitive values appear in confirmation event logs.

## 9. Open questions
- What baseline measures the 20% reduction?
- Which API version owns the confirmation contract?
```

Criterio de salida: documento Markdown de 1-2 paginas, accionable, sin decisiones no sustentadas por la entrada o una decision humana.

Completado

S1 · Paso 5 de 22

Demostrar y versionar el Skill

Tiempo estimado: 15-20 minutos. Demuestra que el Skill es descubrible y ejecutable antes de versionarlo.

1. Listar los skills configurados:

```
find .github/skills -maxdepth 2 -type f
```

Salida esperada: el listado muestra `feature-intake-spec-checklist/SKILL.md` y sus dos ejemplos; esto confirma que el Skill esta en la ruta que Copilot descubre automaticamente en el repositorio.

2. Prompt para ejecutar el Skill (Copilot Chat):

```
Texto
Actua como el Skill feature-intake-spec-checklist
(.github/skills/feature-intake-spec-checklist/SKILL.md).
Usa como entrada el contenido de examples/input.sample.md y genera
UNICAMENTE el documento Output definido en el contrato del Skill.
```

Salida esperada: Copilot Chat responde con un unico documento Markdown con las nueve secciones del contrato (Summary, Scope, Assumptions ... Open questions), coherente con `examples/output.sample.md` y sin inventar datos fuera de la entrada.

3. Commit y tag:

```
git add .github/skills/feature-intake-spec-checklist
```

```
git commit -m "Add basic skill: feature intake to spec checklist
(v0.1.0)"
```

```
git tag skill-feature-intake-v0.1.0
```

Criterio de salida: el Skill es descubrible en `.github/skills/`, produce el documento esperado al invocarlo desde Copilot Chat y queda versionado con commit y tag.

Completado

S1 · Paso 6 de 22

Aplicar el Skill a la HU-01

`Tiempo estimado: 15-20 minutos.` `HU-01.md` no se copia: se genera desde el `HALLAZGOS.md` de tu `workshop-auth-backend` del Dia 1 (filas "Pendiente" sobre el

token).

```
Texto
CONTEXTO
workshop-auth-backend/HALLAZGOS.md es la matriz de hallazgos del Dia 1.

OBJETIVO
Generar workshop-auth-backend/HU-01.md a partir de las filas "Pendiente"
de HALLAZGOS.md relacionadas con el token de autenticacion.

RESTRICCIONES
No inventes hallazgos que no esten en HALLAZGOS.md. No incluyas valores de
tokens, secretos ni credenciales.

CRITERIOS DE ACEPTACION
HU-01.md tiene las secciones Origen, Contexto, Ajustes necesarios, Criterios de
aceptacion y Evidencia esperada. Origen lista los Id (H-xx) y la evidencia
archivo:linea de cada hallazgo usado. Cada criterio de aceptacion es
verificable con una prueba.

FORMATO DE RESPUESTA
Solo el archivo HU-01.md.
```

Solo si tu `HALLAZGOS.md` no tiene hallazgos de token en estado "Pendiente", crea `HU-01.md` y pega esta version de referencia:

```
Texto
# HU-01 - Token firmado con expiracion y claims minimos

## Veredicto
**Usable con ajustes.** La historia conecta seguridad, configuracion, testing y
evidencia.

## Contexto
El servicio de autenticacion expone `POST /auth/login` y devuelve `token`,
`expiresIn` y `user`. La implementacion usa firma HMAC y lee
`AUTH_TOKEN_SECRET`.
```

```
## Ajustes necesarios
- Exigir los claims `sub`, `client`, `iat` y `exp`.
- Obtener `client` desde la solicitud, validarlo y reflejarlo en el token.
- Comprobar que `expiresIn` coincide con `exp - now` usando una tolerancia
documentada.
- Eliminar el secreto por defecto fuera del laboratorio o documentarlo como
exclusivo de laboratorio.
- Evitar logs del token y del secreto.
- Mantener como decision tecnica si se conserva HMAC o se adopta una libreria
JWT.

## Criterios de aceptacion
- Un token valido contiene `sub`, `client`, `iat` y `exp`.
- `expiresIn` es coherente con `exp - now` dentro de la tolerancia definida.
- Login valido, credenciales invalidas y token expirado tienen pruebas.
```

Comprobacion: confirma que el archivo existe y que Copilot Chat puede leerlo desde la raiz del repositorio.

```
test -f HU-01.md && cat HU-01.md
```

Prueba: Genera tu Prompt para usar este skill con el archivo creado (HU-01.md). Puedes tambien usar el siguiente Prompt:

Texto

```
Actua como el Skill feature-intake-spec-checklist
(.github/skills/feature-intake-spec-checklist/SKILL.md).

Lee workshop-auth-backend/HU-01.md y usa el archivo como unica entrada.
Genera UNICAMENTE el documento Output definido en el contrato del Skill,
con sus nueve secciones: Summary, Scope (in/out), Assumptions,
Technical decisions to make, Risks and mitigations, Acceptance criteria,
Validation plan, Suggested tasks by role y Open questions.

No inventes requisitos. Marca como pregunta abierta toda decision que la HU
no confirme. No incluyas valores de tokens, secretos ni credenciales.
```

Revisar el resultado: verifica que la respuesta cubra los claims `sub`, `client`, `iat` y `exp`; la relacion entre `expiresIn` y `exp`; la configuracion segura de `AUTH_TOKEN_SECRET`; ausencia de logs sensibles; pruebas de token, expiracion y claims; y tareas para Tech Lead, Backend y QA. La decision HMAC frente a una libreria JWT debe quedar como una decision tecnica o pregunta abierta, no como un hecho inventado.
Criterio de salida: `HU-01.md` esta versionado en la raiz, el Skill lee el archivo como entrada y produce un documento trazable, sin secretos, con criterios de aceptacion verificables y preguntas abiertas donde falte confirmacion.

Completado
S1 · Paso 7 de 22

Versionar el laboratorio

Tiempo estimado: 5 minutos. Sube todo lo trabajado en S1 (Skill y HU-01.md) y marca el laboratorio con un tag.

Terminal · bash **Básico** Avanzado

```
git add -A
git commit -m "lab02-s1: skill basico feature-intake-spec-checklist + HU-01"
git push origin HEAD
git push origin skill-feature-intake-v0.1.0
git tag lab02-s1
git push origin lab02-s1
```

Criterio de salida: el repositorio remoto contiene `.github/skills/feature-intake-spec-checklist/`, `HU-01.md` y el tag `lab02-s1` apunta al commit final del laboratorio (ademas de `skill-feature-intake-v0.1.0`).

Completado
S2 · Paso 8 de 22

Verificar prerrequisitos: workshops y Skill de S1

Probar comandos y su salida en la terminal web

Tiempo: 5 minutos. Desde `reqflow-workspace/`, confirma que los tres repos y el Skill de S1 existen antes de continuar.

```
test -d workshop-auth-backend && echo "OK: workshop-auth-backend"
```

```
test -d workshop-login-frontend && echo "OK: workshop-login-frontend"
```

```
test -d workshop-requirements-api && echo "OK: workshop-requirements-api"
```

```
test -d workshop-auth-backend/.github/skills/feature-intake-spec-checklist && echo "OK: skill de S1"
```

Si falta alguno de los repos, no clones la plantilla limpia: créalo como repo propio con el paso 1 del README del Dia 1, o parte de la rama `checkpoint/day1`.
Salida esperada: los cuatro `test` confirman presencia.

Probar comandos y su salida en la terminal web

Completado

S2 · Paso 9 de 22

Instalar los cinco Skills desde GitHub

Tiempo: 5 minutos.

El comando es el mismo para los tres repositorios. En el Explorador de VS Code, haz clic derecho sobre la carpeta del repositorio y elige **Abrir en terminal integrada**. Usa Git Bash y comprueba que la ruta termina en el repositorio donde quieres instalar los Skills:

FLUJO	CARPETA DE LA TERMINAL
Autenticación	`reqflow-workspace/workshop-auth-backend/`
Login	`reqflow-workspace/workshop-login-frontend/`
Requisitos	`reqflow-workspace/workshop-requirements-api/`

Ejecuta este único comando dentro del repositorio elegido. Si trabajas con varios repositorios, repítelo en la terminal de cada uno. Ejecutarlo en `reqflow-workspace/` no instala los Skills dentro de sus repositorios hijos.

```
npx --yes skills@1.7.4 add NTTData-Academy/bcp-ai-skill-catalog --
skill discovery-analysis spec-writing architecture-decision issue-
implementation reqflow-definition-of-done -a github-copilot --copy -y
```

Requiere Node.js 22.20 o superior. Si el catálogo es privado, tu cuenta debe tener acceso y Git debe estar autenticado.

- `npx --yes skills@1.7.4` descarga y ejecuta esa versión de la CLI Skills de Vercel. `--yes` evita la confirmación de npx.
- `add NTTData-Academy/bcp-ai-skill-catalog` toma los Skills de ese repositorio de GitHub.
- `--skill` limita la instalación a los cinco nombres que siguen.
- `-a github-copilot` selecciona GitHub Copilot.
- `--copy` copia los archivos sin enlaces simbólicos, para usarlos en Windows.
- `-y` acepta las opciones de instalación de Skills sin preguntas interactivas.

La instalación es local al repositorio donde ejecutas el comando. **Salida esperada:** allí aparecen los cinco directorios `.agents/skills/<nombre-del-skill>/`, cada uno con su `SKILL.md`. Conserva el Skill de S1 en `workshop-auth-backend/.github/skills/feature-intake-spec-checklist/`.

Completado

S2 · Paso 10 de 22

Reconocer los cinco Skills del ciclo SDLC

Tiempo: 5 minutos.

Las instrucciones vienen del repositorio NTTData-Academy/bcp-ai-skill-catalog en GitHub. El comando anterior las copia a tu repositorio. Cada Skill tiene su propio archivo `SKILL.md`; la guía de S2 del portal describe el ejercicio.

En el Explorador de VS Code, abre la carpeta del repositorio donde instalaste los Skills, luego `.agents` → `skills` → el nombre del Skill → `SKILL.md`. Por ejemplo, para autenticación abre `reqflow-workspace/workshop-auth-backend/.agents/skills/discovery-analysis/SKILL.md`.

También puedes leer los archivos originales en GitHub con los enlaces de esta tabla. Si el catálogo es privado, inicia sesión con una cuenta que tenga acceso.

INSTRUCCIONES ORIGINALES EN GITHUB	USO EN EL LABORATORIO
discovery-analysis	Investigar hallazgos con evidencia sin cambiar código
spec-writing	Especificar el cambio aprobado
architecture-decision	Evaluar impacto y registrar decisiones técnicas
issue-implementation	Implementar el issue aprobado con pruebas
reqflow-definition-of-done	Revisar criterios de aceptación y evidencia

Salida esperada: identificas qué Skill usar en cada etapa y encuentras sus instrucciones tanto en tu repositorio como en el catálogo de origen.

Guía de S2 del laboratorio

Completado

S2 · Paso 11 de 22

Verificar los Skills instalados

Tiempo: 5 minutos.

En la misma terminal del repositorio donde ejecutaste la instalación, comprueba los cinco archivos:

Terminal · bash

```
for skill in discovery-analysis spec-writing architecture-decision issue-
implementation reqflow-definition-of-done; do
  if test -f ".agents/skills/$skill/SKILL.md"; then
    echo "OK: $skill"
  else
    echo "FALTA: $skill"
  fi
done
```

Salida esperada: cinco resultados `OK` y ningún `FALTA` por repositorio. Repite esta comprobación en cada repositorio donde instalaste los Skills. Si aparece `FALTA`, revisa la carpeta de la terminal y la salida de la instalación. Copilot reconoce `.agents/skills/` y conserva acceso al Skill de S1 en `.github/skills/`.

Completado

S2 · Paso 12 de 22

Discovery - Encontrar hallazgos y seleccionar un issue candidato

Tiempo: 10-15 minutos. Usa `discovery-analysis` para analizar el producto ReqFlow (multi-repo) y producir una lista corta de hallazgos con evidencia. No modifiques codigo.

Texto

```
Actua como el Skill discovery-analysis
(.agents/skills/discovery-analysis/SKILL.md).

Contexto: estas en reqflow-workspace/ con los tres repos:
- workshop-auth-backend
- workshop-login-frontend
- workshop-requirements-api

Objetivo: produce 3-5 hallazgos con evidencia concreta (archivo:linea o
comando),
impacto, prioridad propuesta, contrato potencialmente afectado y como validarlo.

Restricciones:
- no modificar codigo;
- no crear issues automaticamente;
- marca todo como CANDIDATO y detente para revision humana.
```

Salida esperada: 3-5 hallazgos con evidencia (Estado: CANDIDATO) listos para convertir en Issue tras gate humano.

Completado

S2 · Paso 13 de 22

Spec - Especificar el cambio aprobado

Tiempo: 10-15 minutos. Usa `spec-writing` para convertir el hallazgo/Issue aprobado en una spec verificable (sin diseno ni implementacion).

Texto

```
Actua como el Skill spec-writing
(.agents/skills/spec-writing/SKILL.md).

Fuente: usa como entrada el hallazgo/Issue aprobado (titulo + evidencia +
impacto).
Incluye contratos protegidos de ReqFlow cuando aplique.

Entrega solo la spec en el formato sugerido por el Skill y detente para
revision humana.
```

Salida esperada: spec verificable con alcance, fuera de alcance, criterios de aceptacion y contratos protegidos.

Completado

S2 · Paso 14 de 22

Architecture - Decidir estrategia y pruebas a proteger

Tiempo: 10-15 minutos. Usa `architecture-decision` para evaluar impacto, estrategia incremental y pruebas necesarias antes de implementar.

Texto

```
Actua como el Skill architecture-decision
(.agents/skills/architecture-decision/SKILL.md).

Entrada: spec aprobada.
Salida: impacto, componentes, contratos protegidos, estrategia incremental,
pruebas necesarias, riesgos/trade-offs y decisiones abiertas.

Detente para revision humana antes de implementar.
```

Salida esperada: plan tecnico incremental y testeable (sin reescrituras ni cambios fuera de alcance).

Completado

S2 · Paso 15 de 22

Implementation - Implementar un issue aprobado con evidencia

Tiempo: 15-20 minutos. Usa `issue-implementation` para guiar la implementacion incremental del Issue aprobado, con pruebas y PR auditable.

Texto

```
Actua como el Skill issue-implementation
(.agents/skills/issue-implementation/SKILL.md).

Precondiciones:
- issue aprobado
- spec aprobada
- impacto/arquitectura aprobada

Implementa el cambio minimo necesario, ejecuta pruebas relevantes y prepara
un PR con evidencia real.
```

Salida esperada: implementacion acotada + evidencia de pruebas/validaciones, lista para PR.

Completado

S2 · Paso 16 de 22

Review - Evaluar el cambio con DoD de ReqFlow

Tiempo: 8-10 minutos. Usa `reqflow-definition-of-done` como evaluator final basado en evidencia.

Texto

```
Actua como el Skill reqflow-definition-of-done
(.agents/skills/reqflow-definition-of-done/SKILL.md).
```

```
Evalua el cambio propuesto/implementado para el Issue trabajado en S2.

Para cada criterio responde solo con:
- CUMPLIDO (citar evidencia)
- FALTA EVIDENCIA (decir que falta)
- BLOQUEADO (explicar causa)
- NO APLICA AL ALCANCE (justificar)
- PENDIENTE DE OTRO GATE (indicar cual)

No inventes ejecuciones de CI ni cobertura.
```

Salida esperada: evaluacion DoD con evidencia/citas y una decision explicita de si el cambio puede avanzar a PR.

Completado

S2 · Paso 17 de 22

Versionar el laboratorio

Tiempo: 5 minutos. Sube todo lo trabajado en S2 (Skills instalados, cambios de HU-01/HU-05, matriz de pruebas y evidencia) y marca el laboratorio con un tag.

Ejecuta dentro del repositorio que modificaste. Elige una sola variante. En Básico, ejecuta una línea y revisa su resultado antes de seguir. Avanzado encadena con && y publica la rama y la etiqueta en un solo git push; se detiene si falla una operación. Revisa los archivos preparados antes de crear el commit.

Terminal · bash **Básico** Avanzado

```
git add -A
git commit -m "lab02-s2: skills catalog GitHub + ciclo SDLC (HU-01, HU-05)"
git tag lab02-s2
git push origin HEAD
git push origin lab02-s2
```

Criterio de salida: el repositorio remoto contiene los 5 Skills instalados, los cambios de HU-01 y HU-05, `SDLC-EVIDENCE.md`, y el tag `lab02-s2` apunta al commit final del laboratorio.

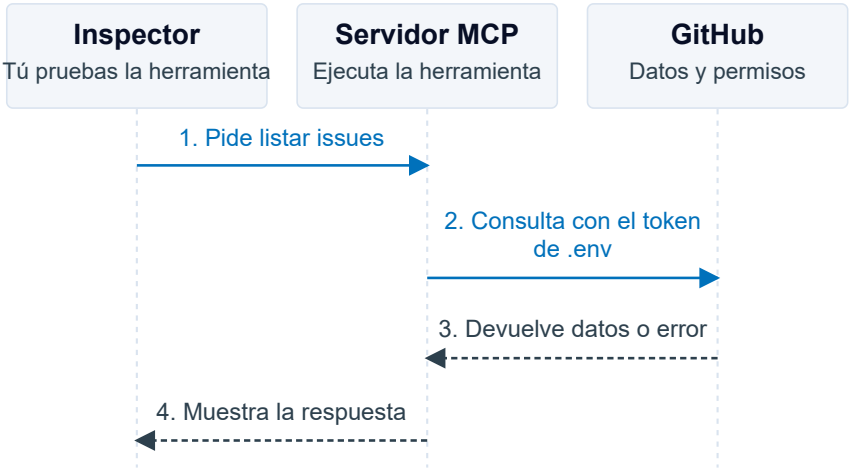
Completado

S3 · Paso 18 de 22

Levantar el MCP local

Usa la carpeta con matrícula y nombre creada por curso. El clon auxiliar bcp-ai-lab-day-02 va al lado de reqflow-workspace/, dentro de esa carpeta. No clones bcp-ai-labs ni arranques el MCP desde el código del portal.

Tiempo: 20-30 minutos. Sigue una petición desde Inspector hasta GitHub, comprende el token del servidor y ejecuta el MCP y su cliente de prueba en dos terminales.



Para qué usamos Inspector

Inspector permite probar las herramientas del MCP manualmente: eliges una, completas sus datos y revisas la respuesta. El **servidor MCP** hace la consulta a **GitHub** usando el token de `.env`.

Después de comprobarlo, conectarás **VS Code con Copilot al mismo MCP**. VS Code hará las peticiones directamente; ya no necesitarás Inspector.

1.1 Preparar `.env` en la raíz del Día 2

Los materiales del Día 2 van **al lado de `reqflow-workspace`**. Esta es la ubicación esperada:

```text
<matricula>-<nombre>/
├── reqflow-workspace/      tus repositorios de ejercicios
└── bcp-ai-lab-day-02/      materiales del Día 2
```

```
├── .env
└── s3_mcp/
```

Si falta `bcp-ai-lab-day-02` : en el explorador de archivos, abre `<matricula>-<nombre>` , la que contiene `reqflow-workspace` . Abre Git Bash en esa carpeta y ejecuta el bloque completo para clonar y entrar en los materiales:

```
>_ Terminal · bash
git clone https://github.com/NTTData-Academy/bcp-ai-lab-day-02.git
cd bcp-ai-lab-day-02
```

Si ya existe: abre Git Bash directamente en `bcp-ai-lab-day-02` y omite el bloque anterior.

Desde `bcp-ai-lab-day-02` , copia la plantilla de configuración:

```
cp -n .env.example .env
```

`-n` conserva tu `.env` si ya existe. Abre ese archivo en tu editor y completa:

```
dotenv
GITHUB_TOKEN=PEGA_AQUI_TU_TOKEN
GITHUB_OWNER=TU_OWNER
GITHUB_REPO=TU_REPO_DE_PRUEBA
AGENT_RECLAIM_ENABLED=false
```

`OWNER` es el usuario u organización; `REPO` , solo el nombre del repositorio. La última variable desactiva una función del servidor que no usaremos. Si `.env` ya existe, conserva su contenido y agrega estos valores.

Para obtener el token de tu sesión de GitHub CLI, ejecuta en tu terminal local:

```
gh auth token
```

Copia el resultado únicamente a `GITHUB_TOKEN` . Si falta la sesión, usa `gh auth login` . Hazlo fuera de capturas o grabaciones. `.env` está excluido de Git; no lo compartas ni lo

incluyas en evidencias. No escribas `$(gh auth token)` dentro del archivo: ahí van valores, no comandos.

1.2 Terminal 1: arrancar el MCP

Antes de arrancar, limpia la caché global de npm. Así `npx` descarga las versiones fijadas y no reutiliza una instalación anterior:

```
npm cache clean --force
```

Después, desde la raíz del Día 2, ejecuta:

```
npx -y mcp-proxy@6.7.20 --host 127.0.0.1 --port 8081 --server stream --shell -- npx -y mcp-github-project-manager@2.3.1 --env-file .env
```

`npx` descarga y ejecuta los paquetes. El proxy abre la conexión HTTP y arranca el servidor, que lee `.env`. `--shell` lanza el servidor a través de tu shell; en Windows es necesario porque `npx` se ejecuta como `npx.cmd`. Espera a que indique que escucha en el puerto **8081** y deja esta terminal abierta. No necesitas construir imágenes.

1.3 Terminal 2: abrir Inspector

Sin cerrar la anterior, abre otra pestaña de terminal y ejecuta:

```
npx -y @modelcontextprotocol/inspector@2.11.0
```

Abre en el navegador la URL completa que imprime Inspector, normalmente en el puerto **6274**. Incluye una clave local que Inspector genera automáticamente; es distinta del token de GitHub. Deja también esta terminal abierta.

DIRECCIÓN	DÓNDE SE USA
URL de Inspector, puerto **6274**	Se abre en el navegador para ver la aplicación de prueba.
`http://localhost:8081/mcp`	Se pega dentro de Inspector para conectar con el MCP de la Terminal 1.

1.4 Navegador: conectar y ver las herramientas

1. En Inspector pulsa **Add Servers → + Add manually**.
2. Completa **Server ID:** `github-local`, **Transport:** `streamable-http` y **URL:** `http://localhost:8081/mcp`.
3. Pulsa **Add** y activa el interruptor de la tarjeta para conectar.
4. Abre **Tools** y busca `discover_tools` y `manage_issues`.

Ver las herramientas comprueba la conexión con el MCP. Todavía no comprueba el acceso a GitHub.

1.5 Probar la consulta del gráfico

1. Selecciona `discover_tools`, elige `domain: issues` y pulsa **Execute Tool** para conocer las acciones disponibles.
2. Selecciona `manage_issues` y completa `action: list`, el mismo owner/repo de tu `.env` y `status: open`. El token no se introduce aquí.
3. Pulsa **Execute Tool**. El MCP consulta GitHub y la respuesta aparece en Inspector.

Una lista de issues, incluso vacía, confirma el acceso al repo para esa consulta. Si recibes un error de permisos, revisa el token y el acceso en GitHub. Guarda la petición y su respuesta como evidencia, sin credenciales.

Continuar con VS Code

Mantén la **Terminal 1** abierta para el Paso 2. Puedes cerrar Inspector con `Ctrl+C` en la **Terminal 2**: el MCP seguirá activo y VS Code se conectará a su misma URL. Cerrar solo la página del navegador no detiene ninguno de los servicios.

Si algo falla

- **No encuentra `.env`:** ejecuta el comando desde la raíz del Día 2 o indica una ruta absoluta después de `--env-file`.
- **Token o repositorio incorrectos:** corrige `.env` y reinicia la Terminal 1. Abre una terminal limpia si antes habías exportado variables `GITHUB_*`.
- **Puerto ocupado:** detén la instancia anterior antes de arrancar otra.
- **Versión antigua o error al descargar:** ejecuta `npm cache clean --force`, cierra la Terminal 1 y vuelve a arrancar el MCP.

- **Inspector no conecta:** confirma que el MCP sigue activo, el transporte es `streamable-http` y la URL termina en `/mcp`.
- **GitHub rechaza la consulta:** revisa permisos y scopes. Para Projects puede hacer falta `gh auth refresh -s project`; después actualiza el token y reinicia el MCP.

Completado

S3 · Paso 19 de 22

Conectar VS Code y descubrir tools

Tiempo: 15-20 minutos. En la raíz de `reqflow-workspace/` (no dentro de un repo) crea `.vscode/mcp.json`, abre Copilot Chat en modo **Agent** estandar y habilita el servidor local.

Texto

```
{
  "servers": {
    "github-local": {
      "type": "http",
      "url": "http://localhost:8081/mcp"
    }
  }
}
```

En **Configure Tools**, verifica que aparezca `github-local` (o el nombre derivado del servidor) y habilita `discover_tools` y `manage_issues`. Para la primera prueba deshabilita temporalmente las Built-in `agent`, `browser`, `edit`, `execute`, `search`, `todo`, `vscode` y `web`. Mantén `read` habilitada: solo puede procesar resultados MCP grandes almacenados por VS Code, no obtiene datos de GitHub.

Texto

```
Usa exclusivamente la herramienta MCP discover_tools.

Busca que herramienta y que accion permiten listar issues de un repositorio.
No uses terminal, archivos locales, web ni ninguna otra herramienta.
```

Salida esperada: identificar `manage_issues` con `action = list`.

Completado

S3 · Paso 20 de 22

Consultar issues y comprobar permisos locales

Tiempo: 20 minutos. Los repos de laboratorio aun no tienen issues (los del backlog, uno por HU, se crean en el paso 04): primero crea un issue temporal con una escritura controlada y despues léelo. Sustituye `<OWNER>` y `<REPO>` por el repositorio configurado. Revisa la llamada propuesta antes de autorizar la escritura.

Texto

```
Usa la herramienta MCP manage_issues.

Ejecuta la accion "create" con estos parametros:
owner: <OWNER>
repo: <REPO>
title: MCP local - write permission test
body: Issue temporal creado para validar permisos del laboratorio MCP.

No uses terminal, GitHub CLI, web ni archivos del workspace.
Antes de ejecutar, muestrame la tool MCP y los parametros que vas a enviar.
```

Ahora que existe al menos un issue, léelo y observa la invocacion MCP y sus parametros como evidencia de que GitHub se consulto mediante el servidor local.

Texto

```
Usa la herramienta MCP manage_issues.

Ejecuta la accion "list" con estos parametros:
owner: <OWNER>
repo: <REPO>
status: open

No uses terminal, GitHub CLI, web, archivos locales ni ninguna otra herramienta.
Resume numero, titulo y estado de cada issue.
```

Evidencia esperada: una llamada equivalente a `Ran Manage Issues` con `action: list`, `owner`, `repo` y `status: open` que devuelve el issue temporal. Luego inicia un chat nuevo y prueba: `Usando el MCP local, consulta los issues abiertos de <OWNER>/<REPO> y resume numero, titulo y estado.` Compara la seleccion automatica de la tool con la invocacion explicita.
Cierra o elimina el issue temporal. Explica donde se aplica una restriccion: tool expuesta, configuracion del servidor, scope del token, permiso de la cuenta o GitHub. La conclusion es: **tool visible != permiso efectivo**.

Completado

S3 · Paso 21 de 22

Conectar GitHub MCP oficial y comparar

Tiempo: 20-25 minutos. Agrega el servidor remoto oficial al mismo `.vscode/mcp.json`. Completa el flujo de autenticacion soportado por tu organizacion; no incluyas tokens en archivos versionados.

Texto

```
{
  "servers": {
    "github-official": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "github-official-readonly": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": {
        "X-MCP-Toolsets": "issues,repos,pull_requests",
        "X-MCP-Readonly": "true"
      }
    }
  }
}
```

Texto

```
Que tools MCP tienes disponibles en github-official?
No ejecutes ninguna; agrupelas por proposito.

Usando exclusivamente github-official, consulta los issues abiertos de
OWNER/REPO y resume numero, titulo
y estado.
No uses terminal, GitHub CLI, busquedas del workspace ni archivos locales.
```

Registra que tool oficial resolvio la lectura y compárala con `manage_issues` del MCP local. Despues crea el backlog que reutilizan los Dias 3 y 4: **un issue por cada HU del dia, en el repo donde vive cada HU** (`<OWNER>/workshop-auth-backend` para HU-01, `<OWNER>/workshop-login-frontend` para HU-05). Titulo: el titulo de la HU. Cuerpo: la seccion Origen (Id `H-xx` y evidencia archivo:linea), los Criterios de aceptacion como checklist y el enlace al PR de S2 si existe. Revisa la tool MCP antes de autorizar cada creacion. Si la escritura es rechazada, conserva el rechazo como evidencia de permisos y crea los issues desde la interfaz web: el Dia 3 los necesita igual.
Crea el Project **ReqFlow** bajo tu owner (una sola vez; los Dias 3 y 4 lo reutilizan), agrega cada issue y completa el campo `Hallazgo origen` con el Id `H-xx` :

Reemplaza TU_OWNER y URL_DEL_ISSUE con tus valores

```
owner="TU_OWNER"
```

```
issue_url="URL_DEL_ISSUE"
```

```
project_number="$(gh project create --owner "$owner" --title ReqFlow --format json --jq .number)"
```

```
gh project field-create "$project_number" --owner "$owner" --name "Hallazgo origen" --data-type TEXT
```

```
gh project item-add "$project_number" --owner "$owner" --url "$issue_url"
```

En **Configure Tools**, comprueba que `github-official-readonly` solo expone lectura y ejecuta una consulta real. Opcionalmente solicita una creacion; si no existe tool de escritura, indícalo y no busques otro mecanismo.
Cierre: `mcp.json` , toolsets y read-only reducen capacidades del cliente, pero no son la unica frontera de seguridad. Aplica scopes minimos, permisos GitHub minimos, rulesets/policies y solo las tools necesarias.

Completado

S3 · Paso 22 de 22

Versionar el laboratorio

Tiempo estimado: 5 minutos. `reqflow-workspace/` no es un repo git: versiona **dentro de cada repo que tocaste**. Guarda la evidencia en `docs/mcp-evidence/` más una **copia** de tu `.vscode/mcp.json` sin tokens; el original se queda en la raíz del workspace.

Desde reqflow-workspace/, en cada repo que cambiaste

Empieza desde reqflow-workspace; la primera línea entra en workshop-auth-backend. Elige una sola variante. En Básico, ejecuta una línea y revisa su resultado antes de seguir. Avanzado encadena con && y publica la rama y la etiqueta en un solo git push; se detiene si falla una operación. Revisa los archivos preparados antes de crear el commit.

Terminal · bash **Básico** Avanzado

```
cd workshop-auth-backend
mkdir -p docs/mcp-evidence
cp ../.vscode/mcp.json docs/mcp-evidence/mcp.json
git add -A
git commit -m "lab02-s3: mcp local + github mcp oficial con permisos explicitos"
git tag lab02-s3
git push origin HEAD
git push origin lab02-s3
```

Cierra además el día (igual que el Día 1), en cada repo que cambiaste: `git tag day2-done` , `git push --follow-tags` .
Criterio de salida: cada repo remoto que cambiaste contiene `docs/mcp-evidence/` (con la copia de `mcp.json` sin tokens ni secretos), el tag `lab02-s3` apunta al commit final del laboratorio y existe el tag `day2-done` .